

# Mini TP — Kiosco: un catálogo con compras

*Integrador de los capítulos 1 a 3 · Programación III · TUP, UTN FRM*

## 1. Objetivo y alcance

Este trabajo práctico integra, en un solo proyecto chico, lo dado en las primeras tres clases: los fundamentos de FastAPI y el protocolo ASGI (capítulo 1), los contratos con Pydantic (capítulo 2), y la ejecución asincrónica (capítulo 3). No es una entidad nueva por cada tema: es un único dominio pequeño donde cada requisito tiene una razón de ser concreta dentro de la historia que cuenta el kiosco.

***Nota:** esto es un mini TP, no el TPI. No hay que persistir nada en una base de datos, no hay autenticación, y no hay colas de tareas en Redis — esos temas todavía no se vieron y se evalúan en otra instancia.*

***Nota:** el dominio tiene una sola entidad (Producto) a propósito. Modelar una relación entre dos entidades distintas — un pedido que referencia a un producto y a un cliente, por ejemplo — todavía no se vio en clase, y por eso queda fuera de este trabajo.*

## 2. El dominio: un kiosco

El kiosco vende productos de un catálogo chico. No hay una segunda entidad para representar la compra: comprar es una operación que se le hace a un producto puntual (identificado por la URL), no un registro nuevo que referencia a otro.

### Producto

| Campo           | Tipo       | Regla                                         |
|-----------------|------------|-----------------------------------------------|
| id              | int        | lo asigna el servidor                         |
| nombre          | str        | obligatorio                                   |
| precio          | Decimal    | mayor a 0 — nunca float (ver 2.7)             |
| stock           | int        | mayor o igual a 0                             |
| stock_reservado | int        | mayor o igual a 0, y no puede superar a stock |
| habilitado      | bool       | por defecto true                              |
| categoría       | str o null | opcional                                      |

***Nota:** stock y stock\_reservado son dos campos del MISMO modelo Producto, no dos entidades distintas. La regla que los relaciona (stock\_reservado no puede superar a stock) es exactamente el caso de la sección 2.3.1: una restricción que no entra en un solo campo.*

### 3. Requisitos obligatorios

Están agrupados por capítulo. Cada uno indica la sección donde se explica el concepto que evalúa — conviene releerla antes de resolverlo.

**Regla general, para todos los requisitos:** toda operación que toque el catálogo (crear, listar, obtener un producto, actualizar, comprar) debe simular la consulta a la base con `await asyncio.sleep(0.05)` antes de tocar los datos en memoria. Ninguna operación responde instantáneamente — eso es, justamente, lo que justifica que cada una sea `async def`.

#### 3.1 — Capítulo 1 (fundamentos de FastAPI y ASGI)

| #  | Requisito                                                                                                                                          | Sección |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| R1 | Todos los handlers, y todos los métodos que acceden al catálogo, están declarados con <code>async def</code> , sin excepción.                      | 1.5     |
| R2 | El catálogo (el repositorio de productos) se recibe en cada handler vía <code>Depends()</code> , en vez de importarse como variable global suelta. | 1.8     |
| R3 | Cada endpoint declara su <code>response_model</code> , de forma que <code>/docs</code> muestre el esquema real de la respuesta.                    | 1.7     |
| R4 | El repositorio se crea una sola vez, en el lifespan de la app, no en cada pedido.                                                                  | 1.9     |

#### 3.2 — Capítulo 2 (contratos)

| #   | Requisito                                                                                                                                                                                                                             | Sección |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| R5  | Producto tiene al menos tres modelos distintos: uno para crear, uno para actualizar y uno para leer (no una sola clase reusada para todo).                                                                                            | 2.5     |
| R6  | El precio se modela con <code>Decimal</code> , no con <code>float</code> .                                                                                                                                                            | 2.7     |
| R7  | Hay una validación sobre un solo campo (con <code>Field</code> ) y una validación que involucra dos campos del mismo modelo a la vez (con un validador de modelo) — que <code>stock_reservado</code> no supere a <code>stock</code> . | 2.3.1   |
| R8  | <code>GET /productos</code> admite paginación (por ejemplo <code>offset</code> y <code>limit</code> ) y la respuesta informa el total de elementos, no solo la página pedida (por ejemplo en un header).                              | 2.9     |
| R9  | Los errores del dominio (producto inexistente, stock insuficiente, producto no habilitado) responden con un formato propio y estable — no alcanza con el <code>{"detail": "..."} por default de FastAPI.</code>                       | 2.10    |
| R10 | <code>PATCH /productos/{id}</code> distingue un campo no enviado de un campo enviado explícitamente como <code>null</code> : el primero no debe modificar el valor existente, el segundo sí.                                          | 2.6     |

**Importante:** ningún modelo puede tener un campo tipado como otro modelo. Si para R8 se te ocurre envolver la lista de productos en un objeto con un campo `items`, es exactamente lo que hay que evitar — por eso el total va en un header y no en un campo del body.

### 3.3 — Capítulo 3 (ejecución asíncrona)

| #   | Requisito                                                                                                                                                                                                                               | Sección        |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| R11 | Al comprar un producto, la verificación de que está habilitado y la de que hay stock suficiente corren de forma concurrente (por ejemplo con TaskGroup), no una atrás de la otra — ninguna de las dos depende del resultado de la otra. | concepto 2 / 4 |
| R12 | La resta de stock queda protegida contra condiciones de carrera: si dos compras del mismo producto llegan al mismo tiempo, ninguna pisa a la otra.                                                                                      | concepto 11    |
| R13 | La notificación de una compra confirmada se dispara como tarea en segundo plano usando la herramienta correcta del framework (no un <code>asyncio.create_task</code> suelto del que nadie se hace cargo).                               | concepto 9     |
| R14 | Hay un endpoint (o un par de endpoints) que demuestra con tiempos de respuesta reales la diferencia entre hacer varias verificaciones de forma secuencial y hacerlas de forma concurrente.                                              | concepto 2     |

*Nota: R11 y R12 se prueban con una sola petición, pero solo se demuestran de verdad con varias peticiones concurrentes — una única compra no puede chocar contra sí misma. Para probarlo en serio hace falta disparar varias compras del mismo producto al mismo tiempo (por ejemplo con `asyncio.gather` desde un script aparte, no con un `for` de `curl`, que las desincroniza).*

## 4. Endpoints de referencia

No es una lista cerrada — nombres de campos o rutas ligeramente distintos no están mal — pero conviene que la forma general (verbo, recurso, de dónde sale cada parámetro) coincida con esto.

| Método y ruta                | Parámetros                                                                          | Devuelve                                       |
|------------------------------|-------------------------------------------------------------------------------------|------------------------------------------------|
| GET /productos               | query: offset (int, default 0), limit (int, default 10)                             | lista de Producto                              |
| GET /productos/{id}          | path: id (int)                                                                      | un Producto, o error 404 normado               |
| POST /productos              | body: ProductoCrear (nombre, precio, stock, stock_reservado, habilitado, categoria) | el Producto creado (201)                       |
| PATCH /productos/{id}        | path: id (int) · body: ProductoActualizar (todos los campos opcionales)             | el Producto actualizado                        |
| POST /productos/{id}/comprar | path: id (int) · body: CompraCrear (cantidad)                                       | el Producto con el stock ya descontado         |
| GET /comparacion/sincrono    | sin parámetros                                                                      | resultados + duracion_ms (versión bloqueante)  |
| GET /comparacion/asincrono   | sin parámetros                                                                      | resultados + duracion_ms (versión concurrente) |

## 5. Entregables

- El código fuente completo (carpeta o repositorio), con requirements.txt.
- Un README breve con las instrucciones para correrlo.
- Evidencia de que R11 y R12 funcionan bajo concurrencia real: puede ser una captura, un log, o un script corto que dispare varias compras a la vez y muestre el resultado.

## 6. Rúbrica

| Bloque                                                     | Puntos |
|------------------------------------------------------------|--------|
| Capítulo 1 — fundamentos (R1 a R4)                         | 20     |
| Capítulo 2 — contratos (R5 a R10)                          | 40     |
| Capítulo 3 — asincronismo (R11 a R14)                      | 30     |
| Calidad general (organización en módulos, nombres, README) | 10     |
| Total                                                      | 100    |

## 7. Qué no se evalúa en este TP

Persistencia en base de datos, autenticación y autorización, colas de tareas en segundo plano con Redis, y relaciones entre entidades distintas quedan fuera de este trabajo a propósito: son temas de capítulos siguientes, y exigir código sobre un concepto que todavía no se vio en clase rompe el orden pedagógico de la cursada. Tests automatizados no son obligatorios, pero suman si se entregan.